

COMP 310 Component Based Software Engineering

Chapter 1 - Introduction to Component Based Software Engineering

1.1. Definitions

1) **Software:**

- Computer programs, procedures, associated documentation and data pertaining to the operation of a computer system (IEEE).
- Software products may be developed for a particular customer or may be developed for a general market

2) **Software engineering (SE)**

- The application of a systematic, disciplined, quantifiable approach to the design, development, operation, and maintenance of software, and the study of these approaches; that is, the application of engineering to software
- Software engineering is an engineering discipline which is concerned with all aspects of software production
- Software engineers should adopt a systematic and organised approach to their work and use appropriate tools and techniques depending on the problem to be solved, the development constraints and the resources available.

3) **Software Component:**

- A component is a software element performing a set of well-defined services; these services satisfy a well identified set of requirements; it has well-defined interfaces and it is managed in the configuration as an element on its own.
- A software component is a software package, a web service, a web resource, or a module that encapsulates a set of related functions (or data).
- A software component is a software element that conforms to a component model and can be independently deployed and composed without modification according to a composition standard.
- A component is a software implementation that can be executed on a physical or logical device. A component implements one or more interfaces that are imposed upon it. This reflects that the component satisfies certain obligations
- A software component may be defined as a unit of composition with contractually specified interfaces and explicit context dependencies only. It is an independent software package that provides functionality via well-defined interface.
- Components are pre-defined pieces of software with a very well establish purpose

4) **Component Based Software Engineering/ Component based software Development**

- Component Component-based software engineering (CBSE) is an approach to software development that relies on software reuse
- It is a reuse-based approach to defining, implementing and composing loosely coupled independent components into systems
- Component-based development is a software engineering approach that focuses on building software systems or components from predefined, independent, and interchangeable units of functionality, called components

5) **Software Reuse**

- Software reuse is the practice of using existing software artifacts, such as code, libraries, frameworks, or design patterns, to create new software systems or components.
- “the systematic use of existing software assets to construct new or modified assets.

Software reuse can reduce development time and cost, increase reliability and performance, and promote consistency and standardization.

6) **Software process**

- A set of activities whose goal is the development or evolution of software. These Generic activities in all software processes are:
- The set of activities and associated results which produce a software product
- Coherent sets of activities for specifying, designing, implementing and testing software systems
- Defines the approach that is taken as software is engineered
- A structured set of activities required to develop a software system

7) **Software Process model**

A simplified representation of a software process, presented from a specific perspective. Examples of process perspectives are

- Workflow perspective - sequence of activities
- Data-flow perspective - information flow
- Role/action perspective - who does what

1.2. Attributes of good software?

The software should deliver the required functionality and performance to the user and should be maintainable, dependable and acceptable.

- a) **Maintainability**- Software must evolve to meet changing needs;
- b) **Dependability**- Software must be trustworthy;
- c) **Efficiency**- Software should not make wasteful use of system resources;
- d) **Acceptability**- Software must be accepted by the users for which it was designed. This means it must be understandable, usable and compatible with other systems.
- e) **Usability** - Software must be usable by the users for which it was designed

1.3. Key challenges facing Software Engineering?

The main challenges in software engineering are; Heterogeneity, delivery and trust.

- a) **Heterogeneity**- Developing techniques for building software that can cope with heterogeneous platforms and execution environments;
- b) **Delivery**- Developing techniques that lead to faster delivery of software;
- c) **Trust**- Developing techniques that demonstrate that software can be trusted by its users.

1.4. Software Development Process;

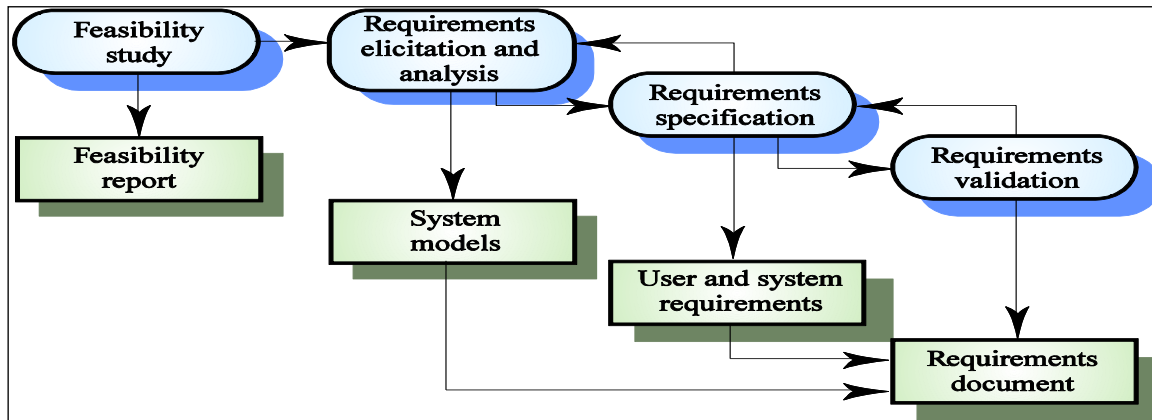
The process is the foundation for software engineering. It holds the technology layers together and enables rational and timely development of computer software. There are four fundamental process activities which are common to all software processes. The activities are:

- 1) **Software specification**: The functionality of the software and constraints on its operation must be defined.
- 2) **Software development**: The software to meet the specification must be produced
- 3) **Software validation**: The software must be validated to ensure that it does what the customer wants
- 4) **Software evolution**: The software must evolve to meet changing customer needs

1.4.1. Software Specification

The process of establishing what services are required and the constraints on the system's operation and development. It is now called Requirements Engineering process and consists of the following phases.

- 1) Feasibility study – done to determine if the proposed system will be cost-effective from a business point of view and can be developed given existing budgetary constraints
- 2) Requirements elicitation and analysis - used to deliver the system requirements.
- 3) Requirements specification - specifying the system requirements
- 4) Requirements validation – checks the requirements for realism, consistency and completeness.



1.4.2. Software Design and Development

The process of converting the system specification into an executable system. It may involve adding formality and detail as the design is developed as well as developing several models of the system at different level of abstraction.

- 1) Software design: The process of converting the system specification into an executable system. Documented as a set of graphical models. Possible structured design methods/models
 - Data-flow model – modelled using data transformations which take place as it is processed.
 - Entity-relation-attribute model – describes the basic entities and their relationships.
 - Structural model;- systems components and their interactions are documented.
 - Object models;- includes the inheritance model of the system etc,
- 2) Software Construction: The construction/development of working, meaningful software through a combination of coding/ components integration, validation, and testing (unit testing)
 - It involves translating a design into a program and removing errors from that program
 - Programming is a personal activity - there is no generic programming process
 - Programmers carry out some program testing on of the code developed, to discover faults in the program and remove these faults in the *debugging process*

1.4.3. Software Validation

Verification and validation is intended to show that a system conforms to its specification and meets the requirements of the system customer. Involves checking and review processes and system testing. System testing involves

- 1) Unit testing: Individual components are tested
- 2) Module testing: Related collections of dependent components are tested
- 3) Sub-system testing: Modules are integrated into sub-systems and tested. The focus here should be on interface testing
- 4) System testing: Testing of the system as a whole. Testing of emergent properties
- 5) Acceptance testing: Testing with customer data to check that it is acceptable

1.4.4. Software Evolution (Software Maintenance)

Software is inherently flexible and can be changed. As requirements change through changing business circumstances, the software that supports the business must also evolve and change. Although there has

been a demarcation between development and evolution (maintenance) this is increasingly irrelevant as fewer and fewer systems are completely new.

1.5. Software Process Models

A (software/system) process model is a description of the sequence of activities carried out in an Software Engineering (SE) project, and the relative order of these activities

A software process model is an abstract representation of a process. It presents a description of a process from some particular perspective. It provides a fixed generic frame-work that can be tailored to a specific project. By changing the process model, we can improve and/or tradeoff

- 1) Development speed (time to market)
- 2) Product quality
- 3) Project visibility
- 4) Administrative overhead
- 5) Risk exposure
- 6) Customer relations,

Facts about Models

- 1) Models do not solve problems; models represent solutions.
- 2) Models do not erase incompatibilities;
- 3) Models communicate intent and form.
- 4) Models are not uniform, either in form or granularity.
- 5) Models do not spring, fully formed, from tools; modeling tools assist in the representation of a constructed model.
- 6) Selecting a modeling tool does not yield a model.
- 7) One type of model does not span all software products or software development processes.

Generic software process models that describes the organization of software processes are;

- 1) The Waterfall model; Separate and distinct phases of specification and development
- 2) Evolutionary development (Prototyping); Specification and development are interleaved
- 3) Spiral Model: an iterative model which includes risk analysis and risk management
- 4) Formal Systems development; A mathematical system model is formally transformed to an implementation
- 5) Reuse-based development; The system is assembled from existing components
- 6) Rational Unified Process (RUP).

1.5.1. Waterfall model

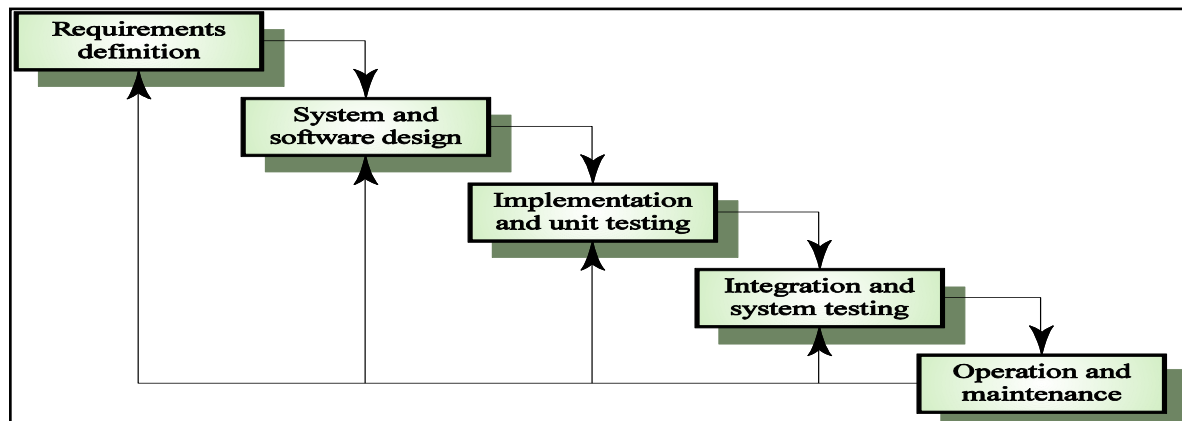
The waterfall model (sometimes called the classic life cycle or the linear sequential was the first explicit model of the software development process. In the waterfall model system design is broken down into a number of sequential stages, with each stage being as a separate box in the model.

- The development process was not allowed to move onto the next stage until work on the previous stage had been completed.
- The outputs from each stage form the inputs of the next stage; therefore each stage had to be completed to maintain the links between the outputs and inputs

Waterfall model phases

- 1) Requirements analysis and definition: system services, constraints and goals are established by consultation with system users

- 2) System and software design: partitions the requirements to either hardware or software systems and establishes an overall system architecture.
- 3) Implementation and unit testing: the software design is utilised as a set of programs or program unit. Unit testing involves verifying that each unit meets its specification
- 4) Integration and system testing: individual program units or program are integrated and tested as a complete system
- 5) Operation and maintenance: the system is put into practical use and maintained



Advantages

- 1) Easy to understand and implement.
- 2) Widely used and known (in theory!)
- 3) Fits other engineering process models: civil, mechanical etc.
- 4) Reinforces good habits: define-before- design, design-before-code
- 5) Identifies deliverables and milestones
- 6) Document driven: People leave, documents don't.
- 7) Works well on large/mature products and weak teams

Drawbacks

- 1) The main drawback of the waterfall model is the difficulty of accommodating change after the process is underway. One phase has to be complete before moving onto the next phase.
- 2) Inflexible partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements. Commitments must be made at early stage in the process.
- 3) The lifecycle is very rigid, and costly for developing a system. Volumes of new documents must be generated and steps repeated if, requirements and specifications need to be revised. Because of the time and cost to repeat the sequence of lifecycle activities, the methodology encourages freezing of specifications early in the development process, discouraging change.
- 4) It is not well suited for unstructured, decision-oriented applications where requirements cannot be immediately visualized. It is difficult to accommodate changes after the process is underway.
- 5) Real projects rarely follow the sequential model that the model proposes.
- 6) The traditional lifecycle is useful for large projects that need formal specification and tight management control over each stage of system building

The model is only appropriate when the requirements are well-understood

1.5.2. Evolutionary Development - Prototyping

It is based on the idea of developing an initial implementation, exposing this to user comment and refining this through many versions until an adequate system has been developed. Specification, development and validation activities are interleaved rather than separate, with rapid feedback across the activities

There are two types of evolutionary development

- 1) Exploratory development: Objective is to work with customers and to evolve a final system from an initial outline specification. Should start with well-understood requirements and the system evolves by adding new features as they are proposed.
- 2) Throw-away prototyping; Objective is to understand the system requirements and develop better requirements definition for the system. The prototype concentrates with those parts of the user requirements that are poorly understood. Should start with poorly understood requirements

An evolutionary approach to software development is often more effective than the waterfall approach in producing systems which meets the immediate need of the customers. This is because the customers specifications can be developed incrementally.

Advantages

- a) Reduces risk of incorrect user requirements
- b) Good where requirements are changing/uncommitted
- c) Regular visible progress aids management
- d) Supports early product marketing

Disadvantages

- a) An unstable/badly implemented prototype often becomes the final product.
- b) Lack of process visibility
- c) Systems are often poorly structured
- d) Requires extensive customer collaboration –
 - Costs customers time/money
 - Needs committed customers
 - Difficult to finish if customer withdraws
- e) May be too customer specific, no broad market
- f) Difficult to know how long project will last
- g) Easy to fall back into code-and-fix without proper requirements analysis, design, customer evaluation and feedback.
- h) Special tools and techniques may be required which allows for rapid development but they may be incompatible with other tools or techniques (e.g. in languages for rapid prototyping)

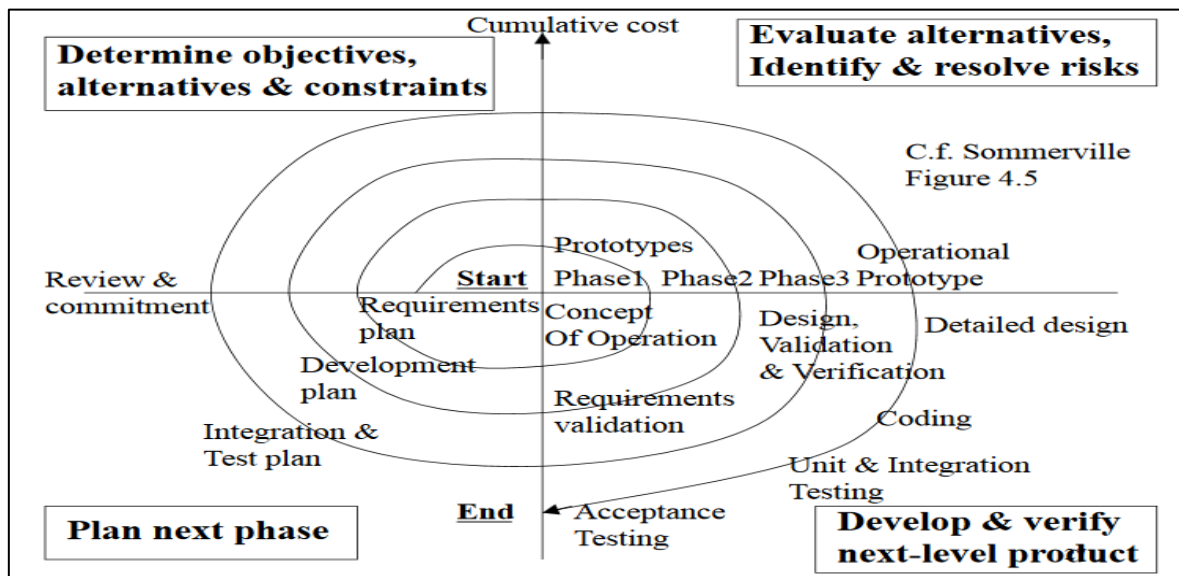
Applicability

- a) For small or medium-size interactive systems
- b) For parts of large systems (e.g. the user interface)
- c) For short-lifetime systems

1.5.3. Spiral Model

Process is represented as a spiral rather than as a sequence of activities with backtracking. Each loop in the spiral represents a phase in the process. No fixed phases such as specification or design - loops in the spiral are chosen depending on what is required. The risks are explicitly assessed and resolved throughout the process. Each loop in the Spiral model is split into four sectors

- 1) Objective setting: Specific objectives for the phase are identified
- 2) Risk assessment and reduction: Risks are assessed and activities put in place to reduce the key risks
- 3) Development and validation: A development model for the system is chosen which can be any of the generic models
- 4) Planning: The project is reviewed and the next phase of the spiral is planned



Advantages

- 1) Realism: the model accurately reflects the iterative nature of software development on projects with unclear requirements
- 2) Flexible: in-corporates the advantages of the waterfall and evolutionary methods
- 3) Comprehensive model decreases risk
- 4) Good project visibility.

Disadvantages

- 1) Needs technical expertise in risk analysis and risk management to work well.
- 2) Model is poorly understood by non-technical management, hence not so widely used
- 3) Complicated model, needs competent professional management. High administrative overhead

1.5.4. Reuse-Oriented Development/ Component Based Development Model

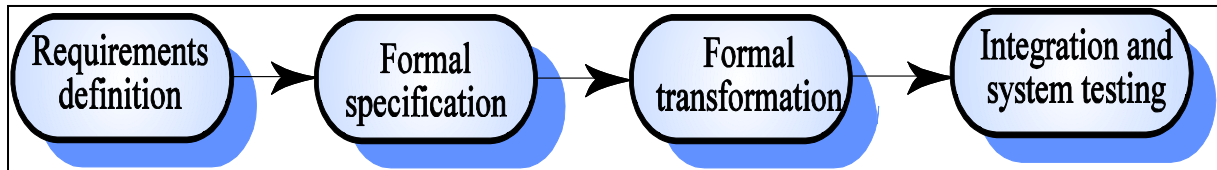
Based on systematic reuse where systems are integrated from existing components or COTS (Commercial-off-the-shelf) systems. This relies on a large base of re-usable software components which can be accessed and some framework for these components.

Process stages includes;

- 1) Requirements analysis and definition- the systems services, constraints and goals are established by consultation with system users. They are then defined in detail and serve as a system specification.
- 2) Component analysis- a search is made for components to implement that specification. Usually there is no exact match, and the components that may be used only provide some of the functionality required.
- 3) Requirements modification – to reflect the available components
- 4) System design with reuse- a framework may be designed or an existing framework be re-used. The designer takes into account the components that are reused and organize the framework to cater to this. Some new software may be designed if no reusable components are available.
- 5) Development and integration – software that can be bought are developed and the components and COTS systems are integrated to create the system
- 6) System testing (validation)- Test the complete system to check for problems that arise from component interaction.
- 7) Operation and maintenance- this is the longest phase. The system is installed and put into practical use. Maintenance involves correcting errors which were not discovered in earlier stages of the life-cycle, improving implementation of system units and enhancing the systems services as new requirements are discovered

1.5.5. Formal Systems Development

Based on the transformation of a mathematical specification through different representations to an executable program. Transformations are ‘correctness-preserving’ so it is straightforward to show that the program conforms to its specification. Embodied in the ‘Cleanroom’ approach to software development



Problems

- Need for specialised skills and training to apply the technique
- Difficult to formally specify some aspects of the system such as the user interface
- Applicability
- Critical systems especially those where a safety or security case must be made before the system is put into operation

1.5.6. Rational Unified Process (RUP)

Hybrid model inspired by Unified Modeling Language (UML) and Unified Software Development Process. A generic component-based process.

RUP Phases

Phases are goal directed and yield deliverables

- Inception – Establish the business case. Identify external entities (actors, systems). Estimate ROI.
- Elaboration – Understand problem domain. Establish architecture, and consider design tradeoffs. Identify project risks. Estimate and schedule project. Decide on build vs. buy.
- Construction. Design, program and test.
- Transition: release a mature version and deploy in real world.

1.6. Challenges of Software development process model

- 1) Poor Requirements – if requirements are unclear, incomplete, too general, and not testable, there may be problems.
- 2) Unrealistic Schedule – if too much work is crammed in too little time, problems are inevitable.
- 3) Inadequate Testing – no one will know whether or not the software is any good until customers complain or systems crash.
- 4) Miscommunication/ poor communication – if developers don't know what's needed or customer's have erroneous expectations, problems can be expected.
- 5) Rigidity and inflexibility, especially in traditional Waterfall models, which makes it difficult to incorporate changing requirements.
- 6) Difficulties with time estimation and project scope definition and management,
- 7) Lack of early risk identification, and challenges in handling complex dependencies and security.
- 8) Models can also suffer from a lack of client involvement or require significant expertise to manage,

1.7. Knowledge Areas (KA) in Software Engineering

Software Engineering Knowledge areas includes;

- 1) Software requirements
- 2) Software Design
- 3) Software Construction
- 4) Software Testing
- 5) Software Maintenance
- 6) Software Configuration Management

- 7) Software Engineering Management
- 8) Software Engineering Process
- 9) Software Engineering Models and Methods
- 10) Software Quality
- 11) Software Engineering Professional Practice

1.7.1. Software Requirements

The Software Requirements KA is concerned with the elicitation, negotiation, analysis, specification, and validation of software requirements. Software requirements express the needs and constraints placed on a software product that contribute to the solution of some real-world problems.

1.7.2. Software Design

Design is defined as both *the process of defining the architecture, components, interfaces, and other characteristics of a system or component* and *the result of [that] process* (IEEE 1991). The Software Design KA covers the design process and the resulting product. The software design process is the software engineering life cycle activity in which software requirements are analyzed in order to produce a description of the software's internal structure and its behavior that will serve as the basis for its construction. It must also describes the components at a level of detail that enables their construction.

1.7.3. Software Construction

Software construction refers to the detailed creation of working software through a combination of detailed design, coding, unit testing, integration testing, debugging, and verification. The Software Construction Knowledge Area (KA) includes topics related to the development of software programs that will satisfy their requirements and design constraints. This KA covers

- 1) Software construction fundamentals (concepts)
- 2) Managing software construction
- 3) Construction technologies
- 4) Practical considerations
- 5) Software construction tools.

1.7.4. Software Testing

Testing is an activity performed to evaluate product quality and to improve it by identifying defects. Software testing involves dynamic verification of the behavior of a program against expected behavior on a finite set of test cases. The Software Testing KA includes the fundamentals of software testing; testing techniques; human-computer user interface testing and evaluation; test-related measures; and practical considerations.

1.7.5. Software Maintenance

Software maintenance involves enhancing existing capabilities, adapting software to operate in new and modified operating environments, and correcting defects. These categories are referred to as perfective, adaptive, and corrective software maintenance. The Software Maintenance KA includes fundamentals of software maintenance (nature of and need for maintenance, categories of maintenance, maintenance costs); key issues in software maintenance (technical issues, management issues, maintenance cost estimation, measurement of software maintenance); the maintenance process; software maintenance techniques (program comprehension, re-engineering, reverse engineering, refactoring, software retirement); disaster recovery techniques, and software maintenance tools.

1.7.6. Software Configuration Management

The configuration of a system is the functional and/or physical characteristics of hardware, firmware, software, or a combination of these. Software configuration management (SCM) is thus the discipline of identifying the configuration of a system at distinct points in time for the purposes of systematically controlling changes to the configuration, as well as maintaining the integrity and traceability of the configuration throughout the software life cycle. The Software Configuration Management KA covers management of the SCM process; software configuration identification, control, status accounting, auditing; software release management and delivery; and software configuration management tools.

1.7.7. Software Engineering Management

Software engineering management involves planning, coordinating, measuring, reporting, and controlling a project or program to ensure that development and maintenance of the software is systematic, disciplined, and quantified. The Software Engineering Management KA covers initiation and scope definition (determining and negotiating requirements, feasibility analysis, and review and revision of requirements); software project planning (process planning, estimation of effort, cost, and schedule, resource allocation, risk analysis, planning for quality); software project enactment (measuring, reporting, and controlling; acquisition and supplier contract management); product acceptance; review and analysis of project performance; project closure; and software management tools.

1.7.8. Software Engineering Process

The Software Engineering KA is concerned with definition, implementation, assessment, measurement, management, and improvement of software life cycle processes. Topics covered include process implementation and change (process infrastructure, models for process implementation and change, and software process management); process definition (software life cycle models and processes, notations for process definition, process adaptation, and process automation); process assessment models and methods; measurement (process measurement, products measurement, measurement techniques, and quality of measurement results); and software process tools.

1.7.9. Software Engineering Models and Methods

The Software Engineering Models and Methods KA addresses methods that encompass multiple life cycle stages; methods specific to particular life cycle stages are covered by other KAs. Topics covered include modeling (principles and properties of software engineering models; syntax vs. semantics vs. invariants; preconditions, post-conditions, and invariants); types of models (information, structural, and behavioral models); analysis (analyzing for correctness, completeness, consistency, quality and interactions; traceability; and tradeoff analysis); and software development methods (heuristic methods, formal methods, prototyping methods, and agile methods).

1.7.10. Software Quality

Software Quality KA includes fundamentals of software quality (software engineering cultures, software quality characteristics, the value and cost of software quality, and software quality improvement); software quality management processes (software quality assurance, verification and validation, reviews and audits); and practical considerations (defect characterization, software quality measurement, and software quality tools).

1.7.11. Software Engineering Professional Practice

Software engineering professional practice is concerned with the knowledge, skills, and attitudes that software engineers must possess to practice software engineering in a professional, responsible, and ethical manner. The Software Engineering Professional Practice KA covers professionalism (professional conduct, professional societies, software engineering standards, employment contracts, and legal issues); codes of ethics; group dynamics (working in teams, cognitive problem complexity, interacting with stakeholders,

dealing with uncertainty and ambiguity, dealing with multicultural environments); and communication skills.

1.8. Professional and Ethical Issues and Responsibility in Software Engineering

Key Ethical and Professional responsibilities in software Engineering includes;

- 1) *Confidentiality*; - Engineers should normally respect the confidentiality of their employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.
- 2) *Competence*; - Engineers should not misrepresent their level of competence. They should not knowingly accept work which is out with their competence.
- 3) *Intellectual property rights*; - Engineers should be aware of local laws governing the use of intellectual property such as patents, copyright, etc. They should be careful to ensure that the intellectual property of employers and clients is protected.
- 4) *Computer misuse*; - Software engineers should not use their technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine, say) to extremely serious (dissemination of viruses).

1.8.1. Professional Bodies and Professional conduct

- 1) Professional Body: Usually a non-profit organization seeking to further a particular profession, the interests of individuals engaged in that profession and the public interest.

Importance of Professional Bodies

The institutions are important because they:

- keep the members up to date with job opportunities within the industry and their local area
- award member for excellence within the field
- will speak to the government on the profession's behalf
- can persuade employers to participate in providing training

As the professional body matures, it is likely to develop a range of functions, of which the following are the most important:

- establishing a code of conduct to regulate the way members of the body behave in their professional lives and a disciplinary procedure to discipline members who breach this code;
- establishing mechanisms for disseminating knowledge of good practice and new developments to its members, typically through publications and conferences but increasingly also through the use of the worldwide web;
- setting standards of education and experience that must be met by people wishing to become members of the body;
- advising government and regulatory bodies about matters within its area of expertise.

- 2) Professional Conduct: A code of conduct sets out the standards of behaviour that members of the body are expected to follow in their professional life.

- A code of practice lays down the best way to practise your profession
- A professional code is a statement of shared values, out of which arises a prescription for right action. As such, these codes share characteristics with moral theory.

1.9. Ethical dilemmas in Software Engineering

Dilemma: *a situation in which there are at least two diametrically opposed actions, each of which supports a desirable outcome.*

Ethical dilemmas in software engineering involve situations where a software professional faces a choice between conflicting values or potential negative consequences, such as

- 1) Balancing data privacy with business profitability,
- 2) Mitigating algorithmic bias,
- 3) Ensuring software security and reliability,
- 4) Designing responsibly, or handling issues like addictive app design, environmental impact, and inclusivity in the workplace.
- 5) Disagreement in principle with the policies of senior management - employer acts in an unethical way and releases a safety-critical system without finishing the testing of the system
- 6) Participation in the development of military weapons systems or nuclear systems
- 7) Social context of Computing
- 8) Professional and ethical responsibilities
- 9) Risks and liabilities of safety-critical systems
- 10) Intellectual property
- 11) Privacy and civil liberties
- 12) Social implications of the Internet
- 13) Computer crime
- 14) Philosophical foundations of Ethics